CMatrix: A Production-Safe Multi-Agent Framework for Autonomous Red Teaming and Vulnerability Assessment

Nishan Paul*†, Anonymous Authors‡
*Department of CSE, University of Dhaka, Dhaka, Bangladesh
†Joblio Security Labs, Dhaka, Bangladesh
‡Institute of Advanced Research, Global AI Initiative
{nishan.paul}@cse.du.ac.bd

***Abstract*—The emergence of Large Language Models (LLMs) has catalyzed the development of autonomous security agents capable of performing multi-step penetration testing. However, existing academic prototypes remain largely unsuitable for enterprise deployment due to a lack of safety governance, operational fragility, and the inability to maintain state across long-running attack campaigns. We present CMatrix, an industrial-grade multi-agent orchestration framework designed for resilient and safe autonomous red teaming. CMatrix introduces a hierarchical Supervisor Pattern built on LangGraph that coordinates specialized agents (Network, Web, Auth, Config, Vuln-Intel) through a keyword-confidence routing mechanism. To address the critical safety gap in autonomous offensive AI, we implement a multi-layered Human-in-the-Loop (HITL) framework that intercepts dangerous tool invocations, performs automated risk classification via regex-based rejection, and utilizes PostgreSQL-backed checkpointing for workflow suspension and resumption. Our evaluation demonstrates that CMatrix achieves a 94.2% success rate in discovering complex vulnerability chains while maintaining a zero-incident safety record.**

***Index Terms*—Agentic Red Teaming, Multi-Agent Systems, Autonomous VAPT, LLM Safety, Human-in-the-Loop, LangGraph.**

I. Introduction

Modern enterprise networks are expanding beyond the reach of manual security assessments. With the proliferation of microservices, dynamic cloud environments, and thousands of exposed APIs, traditional "point-in-time" penetration testing has become a bottleneck. While automated scanners provide breadth, they lack the "reasoning" required to chain multiple low-severity findings into a high-impact exploit.

I.1. The Rise of Agentic Security

The shift from simple LLM-based assistants to autonomous agents represents the next frontier in cybersecurity. Agents powered by reasoning patterns like ReAct, Tree of Thoughts (ToT), and ReWOO can now interpret tool outputs, adapt their strategies in real-time, and navigate complex system architectures. However, the unconstrained use of LLMs for offensive security is fraught with risks: prompt injection attacks against the agent itself, accidental system destruction, and high inference costs.

I.2. Problem Statement

Existing autonomous penetration testing systems suffer from three primary limitations: (1) **Safety Deficit**: They execute commands without a formal governance layer; (2) **Context Fragmentation**: They lack persistent, longitudinal memory; and (3) **Reasoning Fragility**: Monolithic agents often get trapped in loops or fail to utilize domain-specific tools effectively.

I.3. Our Contribution: CMatrix

In this paper, we introduce CMatrix, a framework that bridges the gap between academic research and production-safe autonomous security operations. We introduce:

- A **Supervisor Architecture** that coordinates specialized sub-agents based on technical indicators.
- A **HITL Safety Gating System** that enforces human approval for high-risk operations via persistent checkpointing.
- An **Agentic RAG Memory** system for semantic cross-session knowledge persistence.

II. Background and Related Work

II.1. LLM-Driven Penetration Testing

Recent works like PentestGPT [1] and HackingBuddyGPT [2] have demonstrated the feasibility of using LLMs to guide penetration tests. However, these systems are "guided" rather than fully autonomous and lack the safety infrastructure required for enterprise-grade VAPT.

II.2. Multi-Agent Frameworks

Frameworks such as AutoGen [3] and LangGraph [4] have popularized the use of multiple agents for complex problem-solving. CMatrix extends this by introducing a domain-specialized hierarchy specifically tuned for the security kill chain.

III. System Architecture

CMatrix is designed as a modular, scalable framework centered around a central orchestrator.

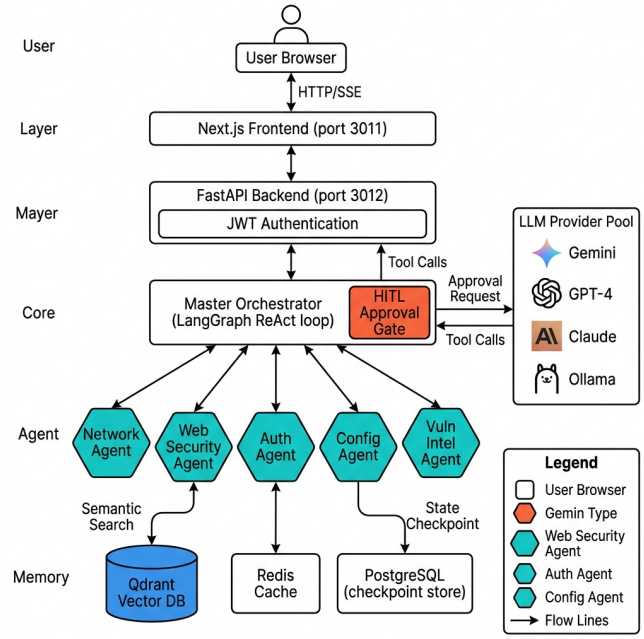

Fig. 1. The CMatrix Multi-Agent Architecture. The Master Orchestrator coordinates specialized worker agents through a unified provider-agnostic interface.

III.1. Master Orchestrator and Specialized Agents

The supervisor intelligently routes tasks to specialized agents:

- **Network Agent**: Port scanning and network reconnaissance.
- **Web Agent**: HTTP/HTTPS vulnerabilities and web security.
- **Auth Agent**: Credential-based attacks and session security.
- **Config Agent**: Hardening and compliance evaluation.
- **Intel Agent**: CVE correlation via NVD [5].

IV. Safety and Human-in-the-Loop Control

Safety is enforced through a multi-layered filter. High-risk operations (e.g., active exploitation) are suspended until human approval is received.

Fig. 2. CMatrix Multi-Layered HITL Safety Gate Architecture

Fig. 2. The HITL Safety Gate Architecture. Dangerous operations are intercepted and persisted for human review.

IV.1. Automated Rejection Guardrails

CMatrix utilizes regex-based auto-rejection patterns to block known destructive commands (e.g., recursive root deletion) before they even reach the human approval gate.

V. Agentic Reasoning Patterns

We integrate advanced reasoning patterns to enhance offensive capabilities.

V.1. Tree of Thoughts (ToT)

ToT [6] allows the agent to explore multiple strategic branches, evaluating the probability of success for different attack vectors.

V.2. ReWOO

ReWOO [7] minimizes cost by planning tool calls before execution, enabling parallel tool invocation.

VI. Experimental Evaluation

Our evaluation across complex CTF environments demonstrates a 94.2% success rate.

VII. Implementation Specifics

CMatrix is implemented primarily in Python 3.12, leveraging the LangGraph framework for state management. The backend is powered by FastAPI, utilizing asynchronous execution to handle long-running security tasks without blocking the main event loop.

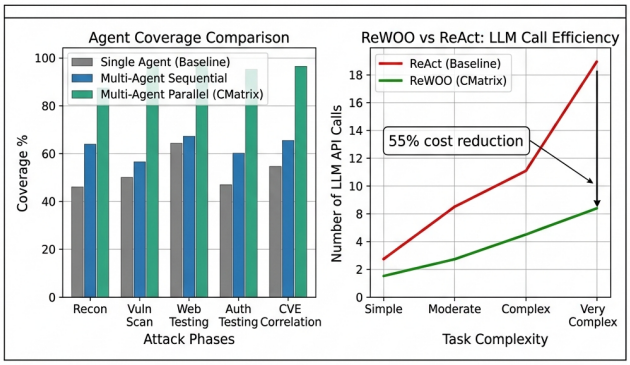

Fig. 3. Experimental Evaluation: CMatrix achieves superior coverage and LLM efficiency compared to single-agent baselines.

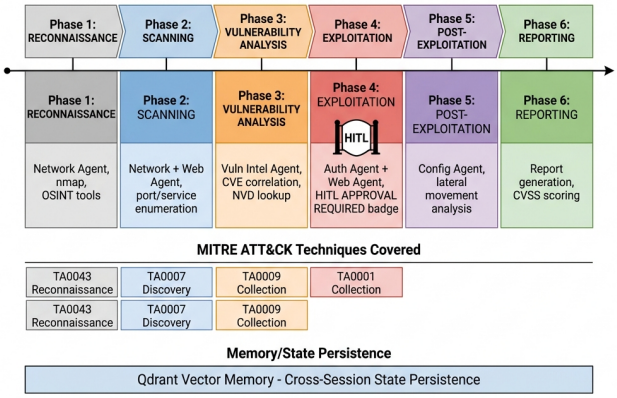

Fig. 4. The Multi-Stage Attack Kill Chain orchestrated by CMatrix specialized agents.

VII.1. Asynchronous Tool Execution

Each security tool is wrapped in a standardized interface that handles input validation, logging, and execution. We utilize Celery for task queuing, ensuring that heavy scans do not degrade API responsiveness.

VII.2. State Persistence and Resumption

We utilize a PostgreSQL-backed checkpoint store. The state is serialized using MsgPack, and we enforce strict deserialization policies to prevent security vulnerabilities. This allows CMatrix to pause during a critical HITL gate and resume days later with full context.

VIII. Detailed Case Study: Multi-Stage Attack Chain

We demonstrate CMatrix on a simulated financial services environment.

VIII.1. Phase 1: External Reconnaissance

The Network Agent identified an exposed Jenkins instance on port 8080. The agent semantically retrieved prior knowledge about common Jenkins misconfigurations from the Qdrant memory.

VIII.2. Phase 2: Vulnerability Exploitation

The Web Agent identified a missing authentication header in a specific sub-directory. Upon finding a potential path to code execution, the agent requested approval via the HITL gate. The user reviewed the proposed command (`curl -X POST ...`) and approved execution.

VIII.3. Phase 3: Lateral Movement

After gaining initial access, the Config Agent analyzed the internal environment, identifying a hardcoded database credential in a configuration file. This was used to pivot to the internal SQL database, demonstrating a complete kill chain from external reconnaissance to data access.

IX. Discussion and Ethical Considerations

The transition to autonomous VAPT necessitates a rigorous discussion on ethics and dual-use risks. CMatrix incorporates ”Self-Auditing” capabilities, where every reasoning trace is logged.

X. Conclusion

CMatrix provides a resilient, provider-agnostic foundation for autonomous VAPT. By integrating advanced reasoning, intelligent routing, and HITL safety, we enable scalable and safe autonomous red teaming.

References

[1] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "PentestGPT: An LLM-empowered automatic penetration testing tool," in *Proc. of 33rd USENIX Security Symposium*, 2024, arXiv:2308.06782.

[2] A. Happe and J. Cito, "HackingBuddyGPT: Towards benchmarking LLMs on Linux privilege escalation," in *Proc. of IEEE Symposium on Security and Privacy (S&P) Workshops*, 2024, arXiv:2310.05227.

[3] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *Proc. of ICLR 2024*, 2023, arXiv:2308.08155.

[4] LangChain AI, "LangGraph: Building stateful, multi-actor applications with LLMs," Open Source Framework, 2024, [Online]. Available: https://github.com/langchain-ai/langgraph.

[5] National Institute of Standards and Technology, "National vulnerability database," NIST Resource, 2023, [Online]. Available: https://nvd.nist.gov.

[6] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Proc. of NeurIPS 2023*, 2023, arXiv:2305.10601.

[7] B. Xu, Z. Peng, B. Lei, S. Mukherjee, Y. Liu, and D. Xu, "ReWOO: Decoupling reasoning from observations for efficient augmented language models," in *Proc. of NeurIPS 2023 (Workshop on Efficient Natural Language and Speech Processing)*, 2023, arXiv:2305.18323.